

Day 4: TypeScript Essentials (Part 2) + Tooling

INTERN NAME

Penumuru Madhu Sudhan
Reddy

TOPIC

Utility Types, Type Guards, tsconfig Strictness &
ESLint Quality

TARGET

Production-Ready & Practical

Overview: Day 4 deepens TypeScript mastery from modeling individual entities to building complete, scalable application architectures. You will learn to construct clean API Data Transfer Objects (DTOs) with built-in utility types, safely narrow polymorphic runtime data using custom type guards, understand why enterprise codebases ban enums in favor of union literals, and configure strict compiler tooling to prevent runtime crashes before deploying code.

1. TypeScript Utility Types in Depth

Utility types are global generic helpers built directly into TypeScript. They transform existing interfaces into new shapes without duplicating code, keeping data models DRY (Don't Repeat Yourself).

A. Partial<T> — HTTP PATCH & Update DTOs

Makes every property in T optional (appends ?). Essential for update endpoints where clients only send changed fields.

PARTIAL<T> IN ACTION

```
interface Project {  
  name: string;  
  budget: number;  
  city: string;  
}  
  
// Without Partial, you'd write a separate interface with optional keys.  
type UpdateProjectDto = Partial<Project>;  
  
const patchPayload: UpdateProjectDto = {  
  budget: 75000000 // Updating only budget!  
};  
console.log("Partial update payload:", patchPayload);
```

TERMINAL OUTPUT:

```
Partial update payload: { budget: 75000000 }
```

B. Required<T> — Strict Creation Validation

Removes all optional flags (?), ensuring every property is strictly provided before writing to a database.

REQUIRED<T> IN ACTION

```
interface SiteConfig {  
  droneInspection?: boolean;  
  maxDailyUploads?: number;  
}  
  
type StrictConfig = Required<SiteConfig>;  
  
const config: StrictConfig = {  
  droneInspection: true,  
  maxDailyUploads: 100  
};  
console.log("Strict config:", config);
```

TERMINAL OUTPUT:

```
Strict config: { droneInspection: true, maxDailyUploads: 100 }
```

C. Pick<T, K> & Omit<T, K> — Data Transformation

Pick: Selects specific fields for lightweight cards or public feeds.

Omit: Excludes sensitive internal fields (e.g. passwords, profit margins) or auto-generated timestamps.

PICK AND OMIT IN ACTION

```
interface TowerRecord {
  id: string;
  name: string;
  totalFloors: number;
  internalBudget: number; // Sensitive field
}

// 1. Pick only display fields
type TowerCard = Pick<TowerRecord, 'name' | 'totalFloors'>;

// 2. Omit sensitive internalBudget
type PublicTower = Omit<TowerRecord, 'internalBudget'>;

const card: TowerCard = { name: "Tower C", totalFloors: 18 };
const publicView: PublicTower = { id: "T-03", name: "Tower C", totalFloors: 18 };

console.log("Public Tower Card:", card);
console.log("Public Tower Record:", publicView);
```

TERMINAL OUTPUT:
Public Tower Card: { name: 'Tower C', totalFloors: 18 }
Public Tower Record: { id: 'T-03', name: 'Tower C', totalFloors: 18 }

D. Record<K, V> — Type-Safe Dictionaries

Creates an object type where keys belong strictly to union K and values match type V. Far safer than { [key: string]: any }.

RECORD<K, V> IN ACTION

```
type TradeCategory = 'Civil' | 'Electrical' | 'Plumbing';

const workerCount: Record<TradeCategory, number> = {
  Civil: 45,
  Electrical: 12,
  Plumbing: 8
};

console.log("Worker Allocation:", workerCount);
console.log("Civil workers count:", workerCount.Civil);
```

TERMINAL OUTPUT:
Worker Allocation: { Civil: 45, Electrical: 12, Plumbing: 8 }
Civil workers count: 45

Utility Type	Transformation	Primary Engineering Use Case
Partial<T>	Makes all properties optional (adds ?)	HTTP PATCH update payloads and filters
Required<T>	Makes all properties mandatory (removes ?)	Ensuring complete records before database insertion
Pick<T, K>	Extracts specified keys K	Lightweight summary views and dropdown models
Omit<T, K>	Removes specified keys K	Stripping sensitive internal properties or auto-ids
Record<K, V>	Maps keys of type K to values of type V	Type-safe lookup maps and category counters

2. Type Narrowing & Custom Type Guards ('is' Predicate)

When external JSON or API payloads arrive, TypeScript cannot know their exact runtime shape. Type narrowing is the process of inspecting values at runtime so TypeScript safely narrows their type inside conditional blocks.

- **typeof:** Primitives (`typeof val === 'string'`).
- **instanceof:** Class instances (`err instanceof Error`).
- **in:** Property existence checks (`'budget' in project`).
- **Custom Type Guards:** Functions returning `val is SpecificType` for complex shapes.

CUSTOM TYPE GUARDS FOR API RESPONSE HANDLING (EXERCISE 2)

```
interface SuccessResponse<T> {
  type: 'success';
  data: T;
  timestamp: string;
}

interface ErrorResponse {
  type: 'error';
  message: string;
  statusCode: number;
}

interface ValidationError {
  type: 'validation';
  fieldErrors: Record<string, string>;
}

type APIResponse<T> = SuccessResponse<T> | ErrorResponse | ValidationError;

// Custom Type Guards using 'is' type predicate:
function isSuccess(res: APIResponse): res is SuccessResponse { return res.type === 'success'; }
function isError(res: APIResponse): res is ErrorResponse { return res.type === 'error'; }
function isValidValidationError(res: APIResponse): res is ValidationError { return res.type === 'validation'; }

function handleApiResponse(res: APIResponse): string {
  if (isSuccess(res)) return `[SUCCESS] Data: ${JSON.stringify(res.data)}`;
  if (isError(res)) return `[ERROR ${res.statusCode}] ${res.message}`;
  if (isValidValidationError(res)) {
    const fields = Object.keys(res.fieldErrors).join(", ");
    return `[VALIDATION FAILED] Invalid fields: ${fields}`;
  }
  return "Unknown response";
}

const res1: APIResponse<{ id: string }> = {
  type: 'success',
  data: { id: 'PRJ-101' },
  timestamp: '2026-10-03T10:00:00Z'
};

const res2: APIResponse<null> = {
  type: 'error',
  message: 'Database connection timeout',
  statusCode: 504
};

const res3: APIResponse<null> = {
  type: 'validation',
  fieldErrors: { budget: 'Budget must be greater than zero' }
};

console.log(handleApiResponse(res1));
console.log(handleApiResponse(res2));
console.log(handleApiResponse(res3));
```

TERMINAL OUTPUT:

```
[SUCCESS] Data: {"id":"PRJ-101"}
[ERROR 504] Database connection timeout
[VALIDATION FAILED] Invalid fields: budget
```

3. Enums vs Union Literal Types (The NirmanIQ Rule)

TypeScript enums are **banned** at NirmanIQ in favor of Union Literal Types. Understanding the technical reasons is essential for clean architecture:

Comparison Factor	TypeScript Enum Banned	Union Literal Types Mandated
Runtime Code	Generates IIFEs and JavaScript lookup objects	Zero runtime code (completely erased on compilation)
Numeric Safety	Numeric enums allow invalid integers (e.g. 999) silently	Strictly forbids any value outside the literal set
JSON Interop	Requires boilerplate translation to and from strings	Direct 1:1 match with JSON strings from databases and APIs
Tree-Shaking	Resists bundler dead-code elimination	Leaves zero traces in the production bundle

UNION LITERAL TYPES IN PRACTICE

```
// Mandated pattern:
type ProjectStatus = 'planned' | 'in_progress' | 'completed';

function getStatusBadge(status: ProjectStatus): string {
  switch (status) {
    case 'planned': return 'Badge: Blue (Planned)';
    case 'in_progress': return 'Badge: Amber (In Progress)';
    case 'completed': return 'Badge: Green (Completed)';
  }
}

console.log(getStatusBadge('in_progress'));
console.log(getStatusBadge('completed'));
```

TERMINAL OUTPUT:
Badge: Amber (In Progress)
Badge: Green (Completed)

4. tsconfig.json Strict Options Explained

The tsconfig.json file governs the TypeScript compiler (tsc). In NirmanIQ services, "strict": true is non-negotiable:

Compiler Flag	Bug / Crash It Prevents	Strict Behavior
strictNullChecks	TypeError: Cannot read property of undefined	Variables cannot be null or undefined unless explicitly typed with null
noImplicitAny	Unintentional dynamic typing and silent bugs	Flags any parameter or variable lacking an annotatable type
strictFunctionTypes	Unsound function argument variance bugs	Enforces strict contravariance on function parameters
strictPropertyInitialization	Accessing uninitialized class fields	Ensures all class properties are set in constructor or declaration
skipLibCheck	Slow build times due to external type definitions	Skips type checking node_modules/**/*.d.ts files (massive build speedup)
noEmit	Accidental clutter of compiled .js files	Only type checks code without writing output files (let Node/Vite run source)

STRICT NULL CHECKS PREVENTING RUNTIME CRASHES

```
interface UserProfile {
  name: string;
  email?: string; // Optional: string | undefined
}

function sendAlert(user: UserProfile): string {
  // If strictNullChecks is active, user.email.toLowerCase() causes COMPILE ERROR.
  // We MUST narrow it first:
  if (user.email) {
    return `Sending alert to: ${user.email.toLowerCase()}`;
  }
  return `No email available for: ${user.name}`;
}

const user1: UserProfile = { name: "Madhu", email: "Madhu@NirmanIQ.com" };
const user2: UserProfile = { name: "Suresh" };

console.log(sendAlert(user1));
console.log(sendAlert(user2));
```

TERMINAL OUTPUT:

```
Sending alert to: madhu@nirmaniq.com
No email available for: Suresh
```

5. ESLint & Prettier Setup: Code Quality Automation

Automated linters detect bugs and enforce engineering standards across teams. ESLint focuses on code correctness and safety, while Prettier enforces consistent visual formatting.

ESLint Rule	Why It's Critical	Bad Pattern	Corrected Pattern
no-var	Prevents function-scope variable hoisting and leaks	var site = "Tower A";	const site = "Tower A";
eql	Prevents type coercion traps (e.g. 0 == "" is true)	if (count == 0)	if (count === 0)
no-explicit-any	Prevents disabling the TypeScript compiler	function log(x: any)	function log(x: unknown)
no-unused-vars	Eliminates dead code and incomplete logic	const score = 100; // unused	Remove or prefix with _score
no-empty	Prevents silently swallowing runtime crashes	try { ... } catch (e) {}	catch (err) { logger.error(err); }

LINT-CLEAN MODERN CODE EXECUTION (EXERCISE 4)

```
function safeCalculateRate(base: number, factor: number): number {
  // 1. Strict equality check (eqeqeq)
  if (factor === 0) {
    return 0;
  }
  // 2. Const declaration instead of var (no-var)
  const rate = (base / factor) * 100;
  return Math.round(rate);
}

function parsePayload(input: unknown): string {
  // 3. Unknown with type guard instead of any (no-explicit-any)
  if (typeof input === 'string') {
    return input.trim();
  }
  return "default";
}

console.log("Calculated Rate:", safeCalculateRate(45, 90));
console.log("Parsed Payload:", parsePayload("  NirmanIQ Site  "));
```

TERMINAL OUTPUT:

```
Calculated Rate: 50
Parsed Payload: NirmanIQ Site
```

6. Complete Architectural Example: CRUD DTO Pattern in Production

Below is the standard DTO architecture used in NirmanIQ microservices, combining Required, Partial, and interface extension to achieve complete end-to-end type safety.

END-TO-END CRUD DTO SERVICE (EXERCISE 1)

```
// 1. Base Project Entity
interface ProjectBase {
  name: string;
  city: string;
  client: string;
  budget: number;
}

// 2. Create DTO (All fields mandatory)
type CreateProjectDto = Required<ProjectBase>;

// 3. Update DTO (All fields optional for PATCH)
type UpdateProjectDto = Partial<ProjectBase>;

// 4. Response Model (Includes system fields)
interface ProjectResponse extends ProjectBase {
  id: string;
  createdAt: Date;
  updatedAt: Date;
}

// In-Memory Repository Simulation
class ProjectRepository {
  private projects: ProjectResponse[] = [];

  create(dto: CreateProjectDto): ProjectResponse {
    const project: ProjectResponse = {
      ...dto,
      id: `PRJ-${this.projects.length + 1}`,
      createdAt: new Date(),
      updatedAt: new Date()
    };
    this.projects.push(project);
    return project;
  }

  update(id: string, patch: UpdateProjectDto): ProjectResponse | undefined {
    const item = this.projects.find(p => p.id === id);
    if (!item) return undefined;
    Object.assign(item, patch, { updatedAt: new Date() });
    return item;
  }

  list(): ProjectResponse[] {
    return this.projects;
  }
}

// Service Execution
const repo = new ProjectRepository();

const created = repo.create({
  name: "Godrej Woods",
  city: "Noida",
  client: "Godrej Properties",
  budget: 50000000
});

console.log("Created Project:", created.id, "-", created.name);

const updated = repo.update(created.id, { budget: 55000000 });
console.log("Updated Budget:", updated?.budget);
console.log("Total Projects in DB:", repo.list().length);
```

TERMINAL OUTPUT:

```
Created Project: PRJ-1 - Godrej Woods
Updated Budget: 55000000
Total Projects in DB: 1
```